

CampusGig

A University-Restricted Gig Marketplace for Thapar Institute of Engineering and Technology

Submitted by

1024240045 Shashwat Narwal

1024240051 Mohd Arham

1024240060 Krish Gakhar

1024240067 Ineshvir Singh

Group: 2 BE Third Year, AIML

Submitted to

Dr. Anushka

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

September 2026

Contents

- 1 Introduction 2**
 - 1.1 Background and Context 2
 - 1.1.1 Problem Statement 3
 - 1.2 Project Objectives 3
- 2 Methodology and Design 5**
 - 2.1 System Architecture 5
 - 2.1.1 High-Level Design 5
 - 2.1.2 Technical Stack 10
 - 2.2 Implementation Details 10
 - 2.2.1 Module 1: Core Functionality 12
 - 2.2.2 Module 2: Secondary Features 16
- 3 Results and Evaluation 20**
 - 3.1 Testing Methodology 20
 - 3.2 Performance Metrics 21
 - 3.3 Analysis 22
- 4 Conclusion 24**
 - 4.1 Summary of Work 24
 - 4.2 Key Findings 24
 - 4.3 Future Work and Recommendations 25
 - 4.4 Final Remarks 26
- Bibliography 27**

CHAPTER 1

Introduction

1.1 Background and Context

Students on a residential campus already trade small pieces of work between themselves. Someone needs a employer designed before a society deadline, someone else wants a lab report proofread, a club needs a photographer for one evening, a fresher wants two hours of tutoring before a quiz. At present this happens through WhatsApp groups, Instagram stories and word of mouth. That works, but only for people who already know the right group, and it leaves no record of who agreed to what.

CampusGig is a web application that gives this activity a proper home. It is a gig marketplace whose membership is restricted to a single institution: only an account registered with an `@thapar.edu` address can sign in and use it. A student can post a piece of work with a budget and a deadline, other students can browse the open postings and apply to the ones they want, and the person who posted decides who gets it.

The restriction to institutional email is the feature that separates this from a general freelancing site. On a public platform the hard problem is trust between strangers. Inside a campus, the university has already done the identity work: it issued the email address, and it can revoke it. Building on top of that means the platform inherits an identity guarantee it does not have to create for itself, and both sides of a gig know that the other person is a real student who can be found on campus.

The work reported here covers the first three milestones of an eight-milestone plan. Those three deliver a running system: registration, sessions, gig posting, gig browsing, applications, and a student profile. The remaining milestones, which add contracts, escrow, a payment ledger, deliverable uploads, disputes and reputation, have been designed in detail and modelled in UML but have not been built. Section 3.3 and Chapter 4 are explicit about which parts of the design have code behind them.

1.1.1 Problem Statement

Informal campus gig exchange has four concrete problems, and CampusGig is scoped around them.

There is no shared place to look. A student looking for work has to be in the right group chat at the right time. A student needing work done has to broadcast into several groups and hope. Neither side has a searchable list of what is currently on offer.

Anyone can join a group chat. A WhatsApp group has no membership test. Someone from outside the institute, or someone impersonating a student, can post in it. There is no cheap way to confirm that the person offering to do the work is actually enrolled.

Nothing is recorded. Once a message scrolls away, the agreed scope, price and deadline are gone. If the two people later disagree about what was promised, there is no record to check, and no history that a third student could use to judge whether someone is reliable.

Payment is unprotected. Money changes hands by UPI on trust alone. The person who pays first can be left with nothing; the person who works first can be left unpaid. Neither party has any recourse.

The system described in this report addresses the first three problems directly. It provides a single searchable board of open gigs, it enforces institutional membership in the backend, and it stores every gig, application and status change in a database instead of a chat log. The fourth problem, payment protection, is the subject of the escrow and ledger design presented in Chapter 2 but is not implemented in this release, and the interface says so explicitly rather than implying that money is being handled.

1.2 Project Objectives

1. **Objective 1:** Restrict all access to verified institutional accounts. Registration must accept an address if and only if its domain is exactly `thapar.edu`, the check must run in the backend rather than only in the browser, and it must reject near-miss domains such as `fakethapar.edu` and `thapar.edu.fake.com`.
2. **Objective 2:** Provide authenticated sessions that can actually be ended. A login must create a server-side session delivered in an `HttpOnly` cookie, logout must invalidate that session on the server, and an expired or forged cookie must be rejected.

3. **Objective 3:** Deliver a working gig board. A signed-in student must be able to post a gig, browse the open ones with search, category, budget and deadline filters, sort and page through the results, and open a single gig for detail.
4. **Objective 4:** Implement the application workflow end to end. A student must be able to apply to a gig they did not post, the employer must be able to accept, decline or mark the work complete, and accepting one applicant must close the gig and decline the rest.
5. **Objective 5:** Give every student a profile with a picture, a resume in PDF form and a short description, with uploads validated by file content rather than by what the browser claims the file is.
6. **Objective 6:** Produce a UML design for the complete platform, including the parts not yet built, so that the contract, escrow, ledger and dispute milestones can be implemented against a model rather than improvised.
7. **Objective 7:** Cover every rejection path in the specification with an automated test, and run those tests against real PostgreSQL with the real database migrations applied.

These objectives are specific and each one is checkable. Objectives 1 to 5 are verified by the test suite described in Chapter 3; Objective 6 is verified by the diagrams presented in Chapter 2; Objective 7 is verified by the composition of the suite itself. They are achievable within a semester because they were sized against the milestone plan in the repository, and they are relevant because each one maps to one of the four problems stated above. The time bound is the mid-semester evaluation, which is the state of the work this report describes.

CHAPTER 2

Methodology and Design

2.1 System Architecture

CampusGig is built as two deployable pieces and a database. A React single-page application runs in the browser, a FastAPI service handles every request that touches data, and PostgreSQL stores it. The browser never talks to the database, and it never holds a credential: the session travels as an `HttpOnly` cookie that JavaScript in the page cannot read. Uploaded avatars and resumes are written to a directory on disk identified by the `MEDIA_ROOT` setting and served back under a `/media` path.

Inside the backend, code is organised into four layers and dependencies point in one direction only. Routers in `api/` deal with HTTP and nothing else: they read request bodies, declare their dependencies and return response models. Services in `services/` hold the business rules and do not import anything from FastAPI, so a rule such as “only the employer may accept an applicant” is testable without constructing a request. Repositories in `repositories/` contain every SQLAlchemy query in the project. Models in `models/` are the only place where a table is declared. A router never builds a query, and a service never raises an `HTTPException`: it raises a domain error, and a single handler registered in `main.py` maps that error to a status code and a response body. This is the repository and service layering described by Fowler [Fow02], applied at the scale this project actually needs.

The practical payoff of the layering shows up in two places. Every error the API returns has the same shape, `{ detail: { code, message } }`, because there is exactly one place that builds it, which means the frontend has one parser rather than a special case per endpoint. And moving from local disk to object storage later is a change to `storage_service.py` alone, because that is the only module in the codebase that touches the filesystem.

2.1.1 High-Level Design

The scope of the platform, including the parts scheduled for later milestones, is captured in the use case diagram in Figure 2.1. It identifies four human actors and three external ones. A *Verified Student* is the base actor; *employer* and *Worker*

are specialisations of it, which is the important modelling decision here. They are capacities, not account types. The same person is a employer on a gig they created and a worker on one they applied to, so the system does not ask anyone to choose a role at signup. The *Administrator* exists for dispute resolution and takedowns. The external actors are the institutional mail service, an object store for deliverables, and a scheduler that fires time-based events such as the auto-release of a confirmation window.

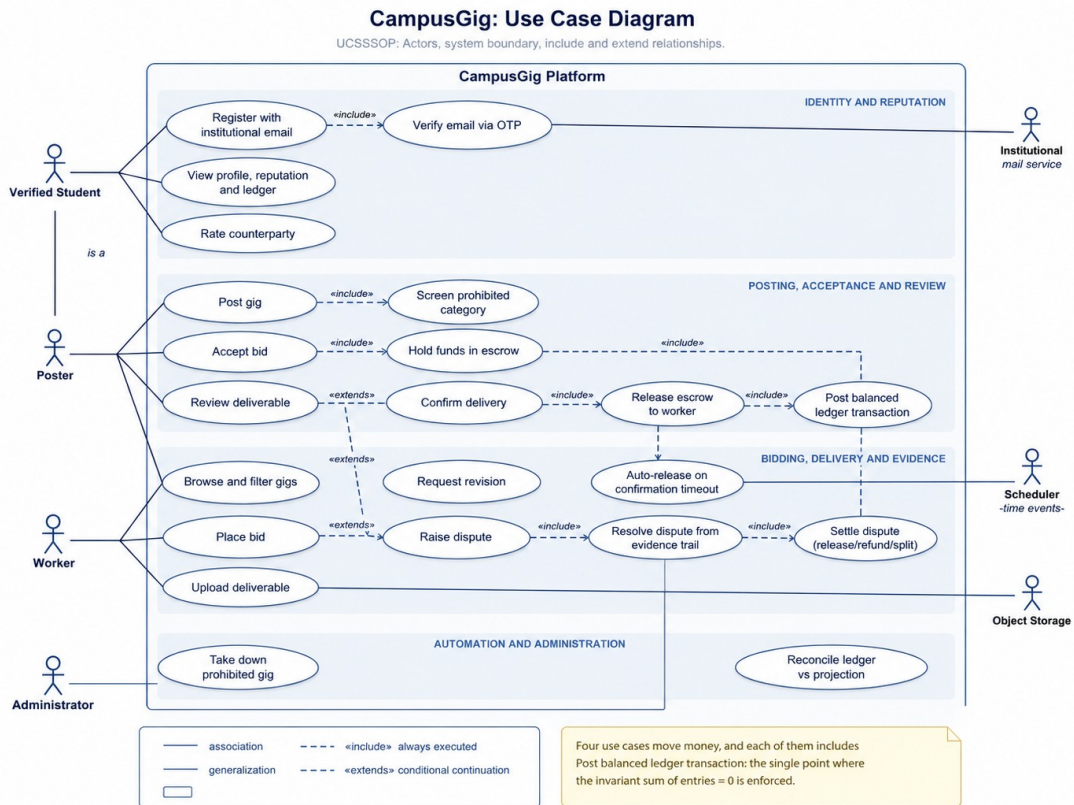


Figure 2.1. Use case diagram for the CampusGig platform, showing actors, the system boundary, and the **include** and **extend** relationships between use cases.

The dashed **include** arrows mark steps that always run as part of the parent use case, and the **extend** arrows mark conditional continuations. Requesting a revision extends reviewing a deliverable because it only happens sometimes; screening a prohibited category is included in posting a gig because it always runs. Four use cases move money, and every one of them includes *Post balanced ledger transaction*, which is the single point where the accounting invariant is enforced.

Of the use cases in Figure 2.1, the current implementation covers registration, browsing and filtering gigs, posting a gig, and viewing a profile. *Verify email via OTP* was built and then deliberately removed, which is discussed in Section 2.2.1. Applying to a gig is implemented as a simplification of *Place bid*: a student applies at the

posted budget and there is no counter-offer. Everything involving escrow, deliverables, disputes, ratings and reconciliation is designed but not coded.

The domain model is split across two class diagrams because a single sheet would not be readable. Figure 2.2 covers the entities a student interacts with.

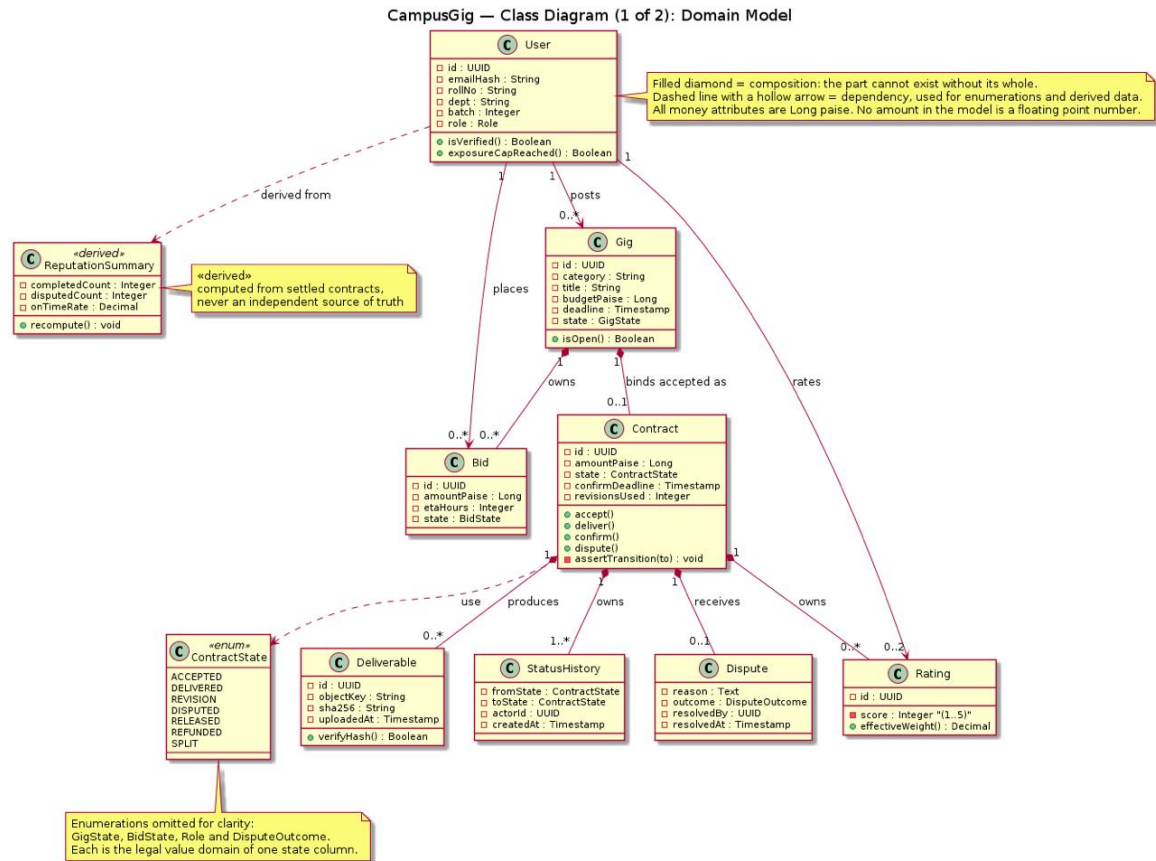
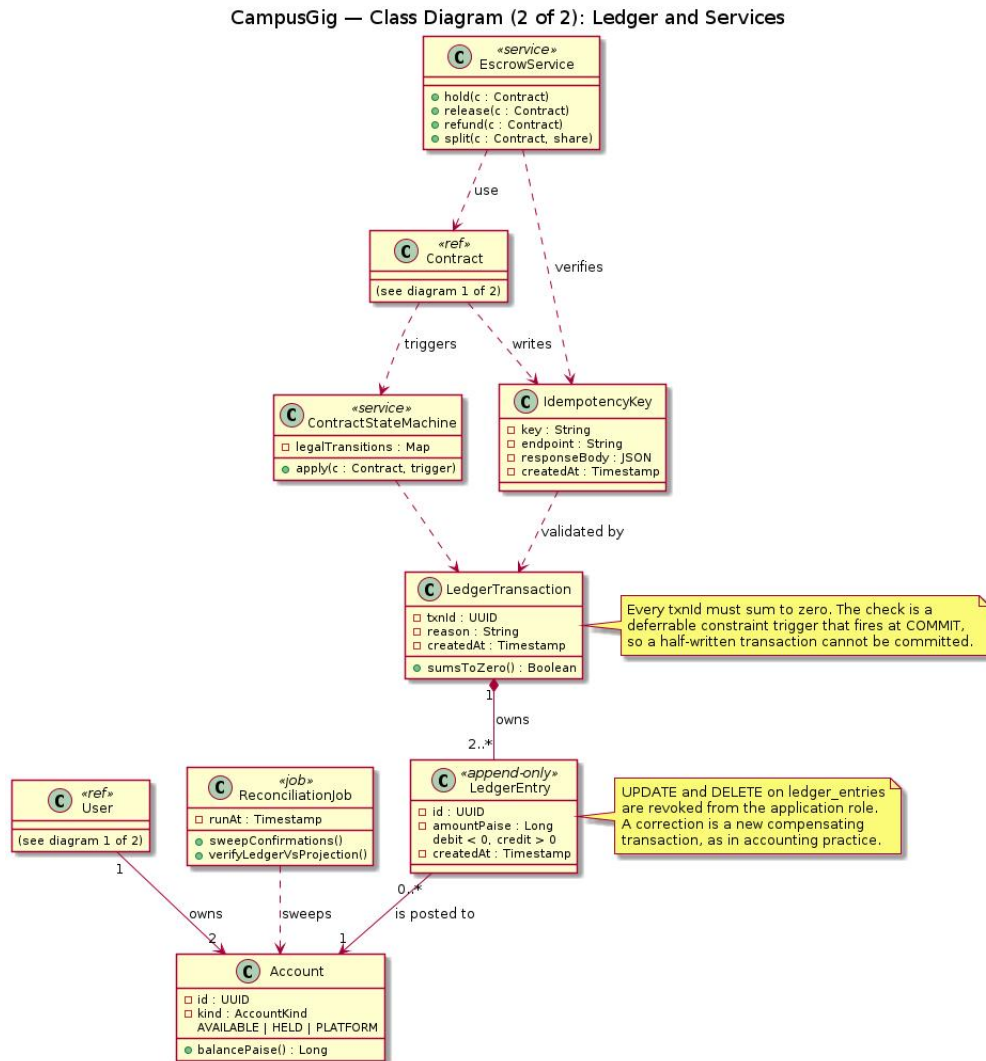


Figure 2.2. Class diagram, part 1 of 2: the CampusGig domain model.

User sits at the centre and holds identity fields only. A user posts many gigs, places many bids, and can hold ratings. A Gig binds at most one accepted Contract, which is where a gig turns into an obligation. A contract owns its Deliverable records, its StatusHistory entries, at most one Dispute, and up to two Rating entries, one from each side. The filled diamonds are compositions: a status history row has no meaning without the contract it belongs to.

Two details in this diagram carry real weight. ReputationSummary is marked «derived», which records that it is recomputed from settled contracts and is never an independent source of truth, so it can be rebuilt if it drifts. And every money attribute is declared as Long paise. No amount anywhere in the model is a floating point number, because binary floating point cannot represent a decimal currency amount exactly and rounding error in a ledger is not recoverable. The implemented code follows the same rule using PostgreSQL numeric(10,2) for the gig budget.

Figure 2.3 covers the accounting and service classes.

**Figure 2.3.** Class diagram, part 2 of 2: ledger entities and the services that write to them.

The design here is standard double-entry accounting. A **LedgerTransaction** owns two or more **LedgerEntry** rows, each of which is posted to an **Account**. Each user owns two accounts, one **AVAILABLE** and one **HELD**, and there is a platform account. Money moving into escrow is not a flag on a row; it is a transaction that debits the employer's available balance and credits their held balance, so the total never changes and a balance is always the sum of the entries posted to an account.

Two notes on the diagram record the constraints that make this safe. First, every transaction must sum to zero, checked by a deferrable constraint trigger that fires at commit, so a half-written transaction cannot be committed. Second, **LedgerEntry** is marked **«append-only»**: update and delete are revoked from the application's database role, and a correction is a new compensating transaction rather than an edit, which is how correction works in accounting practice. **IdempotencyKey** stores

the response body against a client-supplied key so that a retried request returns the original answer instead of holding funds a second time. None of these classes exist in the current codebase; they are the design for Milestone 5.

The lifecycle that ties the money design together is the contract state machine in Figure 2.4.

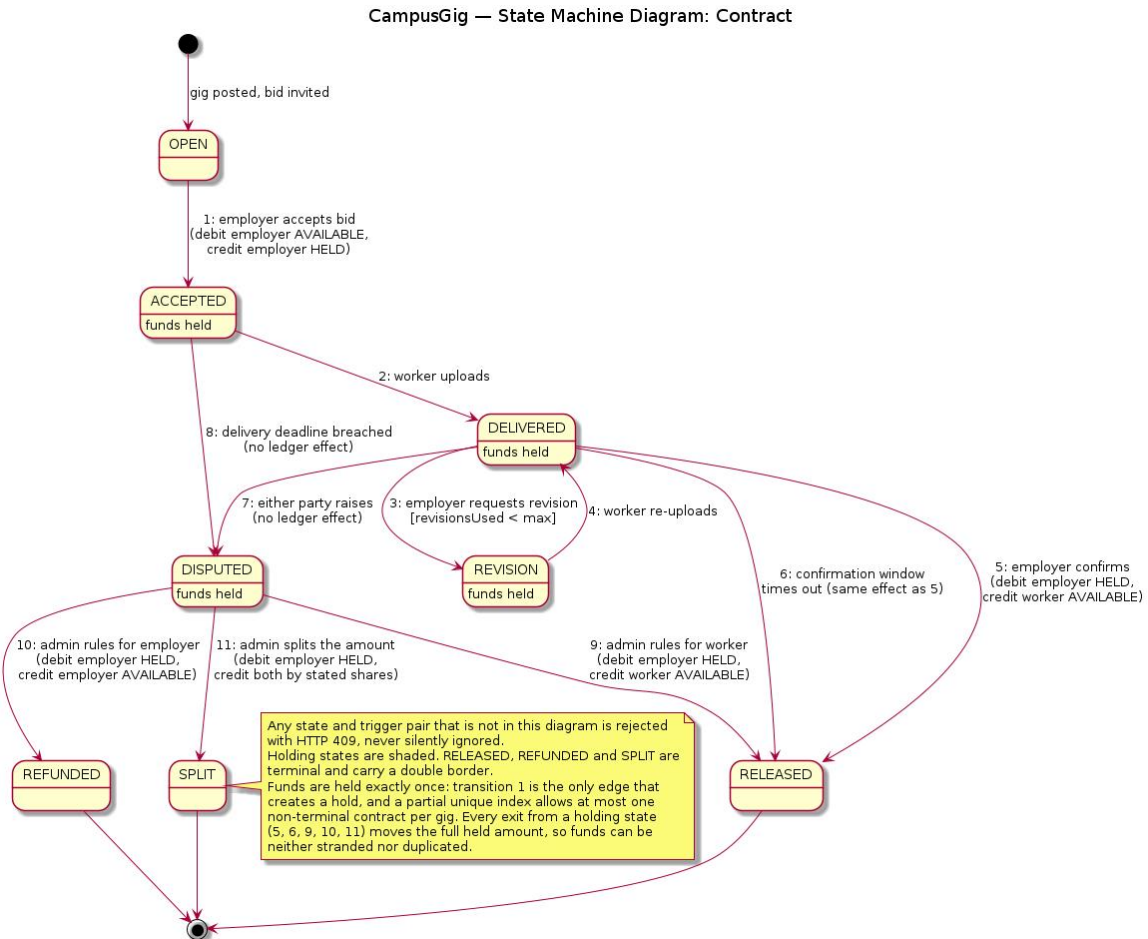


Figure 2.4. State machine diagram for a contract, with the ledger effect of each transition.

A contract starts **OPEN** and ends in one of three terminal states: **RELEASED**, **REFUNDED** or **SPLIT**. Each labelled transition names its ledger effect, so the diagram doubles as the specification for what **EscrowService** must write. Transition 1 is the only edge that creates a hold, and transitions 5, 6, 9, 10 and 11 each move the full held amount out, which is what guarantees that funds are neither stranded in escrow nor released twice. The note on the diagram states the rule the implementation must follow: any state and trigger pair not drawn here is rejected with HTTP 409 rather than silently ignored.

The application status flow that is implemented today is a much smaller version of the same idea. Its legal transitions live in the **ALLOWED_TRANSITIONS** dictionary in

`application_service.py`: `APPLIED` may become `ACCEPTED` or `REJECTED`, `ACCEPTED` may become `COMPLETED`, and nothing leaves `REJECTED` or `COMPLETED`. Anything else raises `InvalidStatusTransition`, which the error handler turns into a 409. The structure is deliberately the same as the diagram so that replacing it with the full contract machine later is an extension rather than a rewrite.

2.1.2 Technical Stack

- **Framework/Language:** Python 3.12 with FastAPI for the backend API, and TypeScript with React 18 for the browser application [Ram24; 24b]. FastAPI was chosen because it derives request validation and the OpenAPI document from the same type annotations the code already needs, so the interactive API documentation at `/docs` cannot drift from the endpoints.
- **Database:** PostgreSQL 15 or later [24a], accessed through SQLAlchemy 2.x with Alembic for migrations [Bay24]. PostgreSQL is not interchangeable here: the schema uses `citext` for case-insensitive email uniqueness, `gen_random_uuid()` for primary keys, and a generated column that derives `email_hash` from `email` inside the database so the two cannot fall out of step.
- **Tools:** Vite as the frontend build tool and development server, pytest for backend tests, Vitest with React Testing Library for frontend tests, Ruff for Python linting, ESLint and the TypeScript compiler for the frontend, and Docker Compose for a local PostgreSQL instance.
- **Libraries:** Pydantic v2 for request and response schemas, `bcrypt` for password hashing [PM99], `python-multipart` for file uploads, React Router for client-side routing, and Framer Motion for page, list and dialog animation. There is no HTTP client library on the frontend: a single `request()` wrapper over the browser's native `fetch` is the only code that knows the backend URL.

2.2 Implementation Details

The two activity diagrams describe the intended end-to-end workflow of the platform. They are the clearest statement of what the finished system is meant to do, and they make it easy to point at exactly where the current implementation stops. Figure 2.5 covers everything up to the moment a gig is awarded.

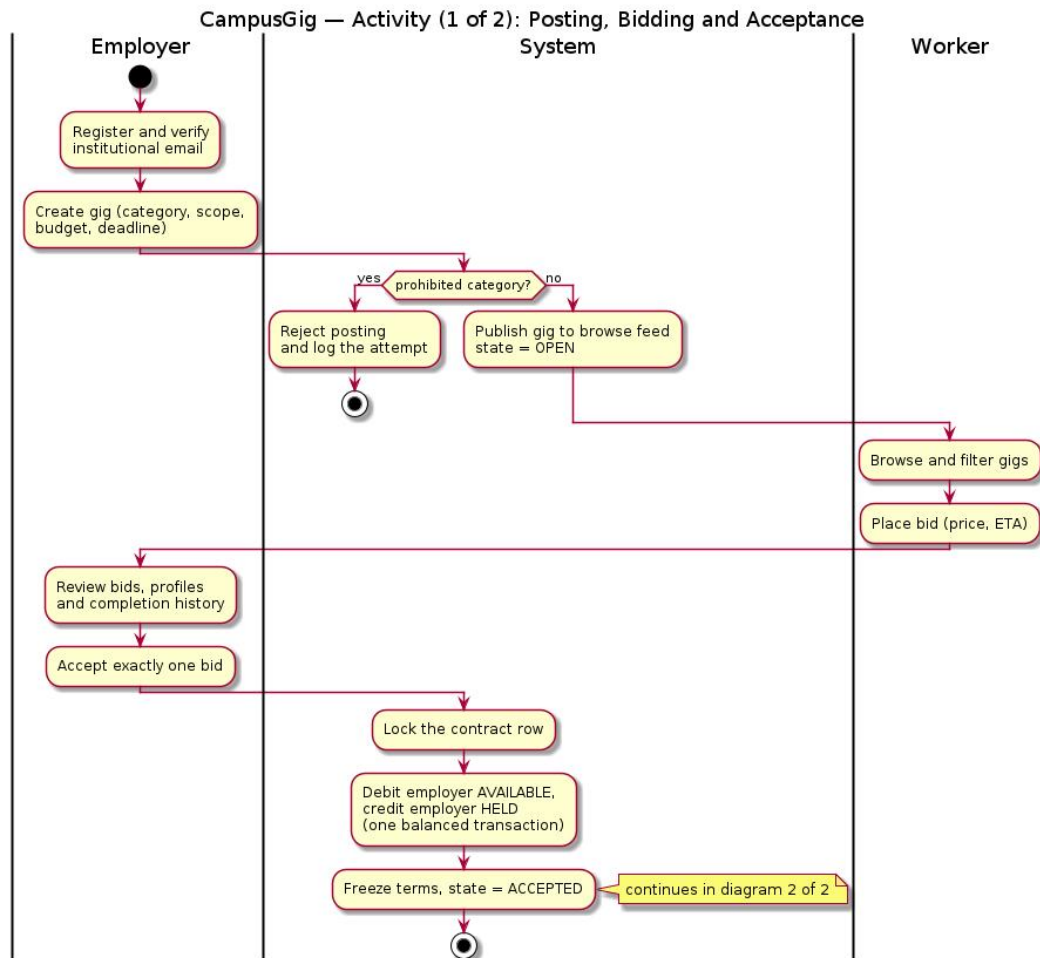


Figure 2.5. Activity diagram, part 1 of 2: posting, bidding and acceptance, partitioned by responsibility.

The swimlanes separate what the employer does, what the system does automatically, and what the worker does. Reading down the diagram: a student registers, creates a gig, the system screens the category and either rejects the posting or publishes it in the `OPEN` state, a worker browses and places a bid, the employer reviews bids and accepts exactly one, and the system then locks the contract row, writes one balanced ledger transaction moving the amount from available to held, and freezes the terms at `ACCEPTED`.

The implemented system follows the left and right lanes of this diagram closely and stops before the system lane’s last three activities. Registration, gig creation, publication as `OPEN`, browsing with filters, and the employer’s decision are all built. Category screening is not: the category is a validated enum, but there is no prohibited-category rule. Placing a bid is implemented as applying at the posted budget. Accepting an applicant does close the gig and reject the others, but no row is locked and no ledger entry is written, because the ledger does not exist yet.

Figure 2.6 continues from the acceptance point.

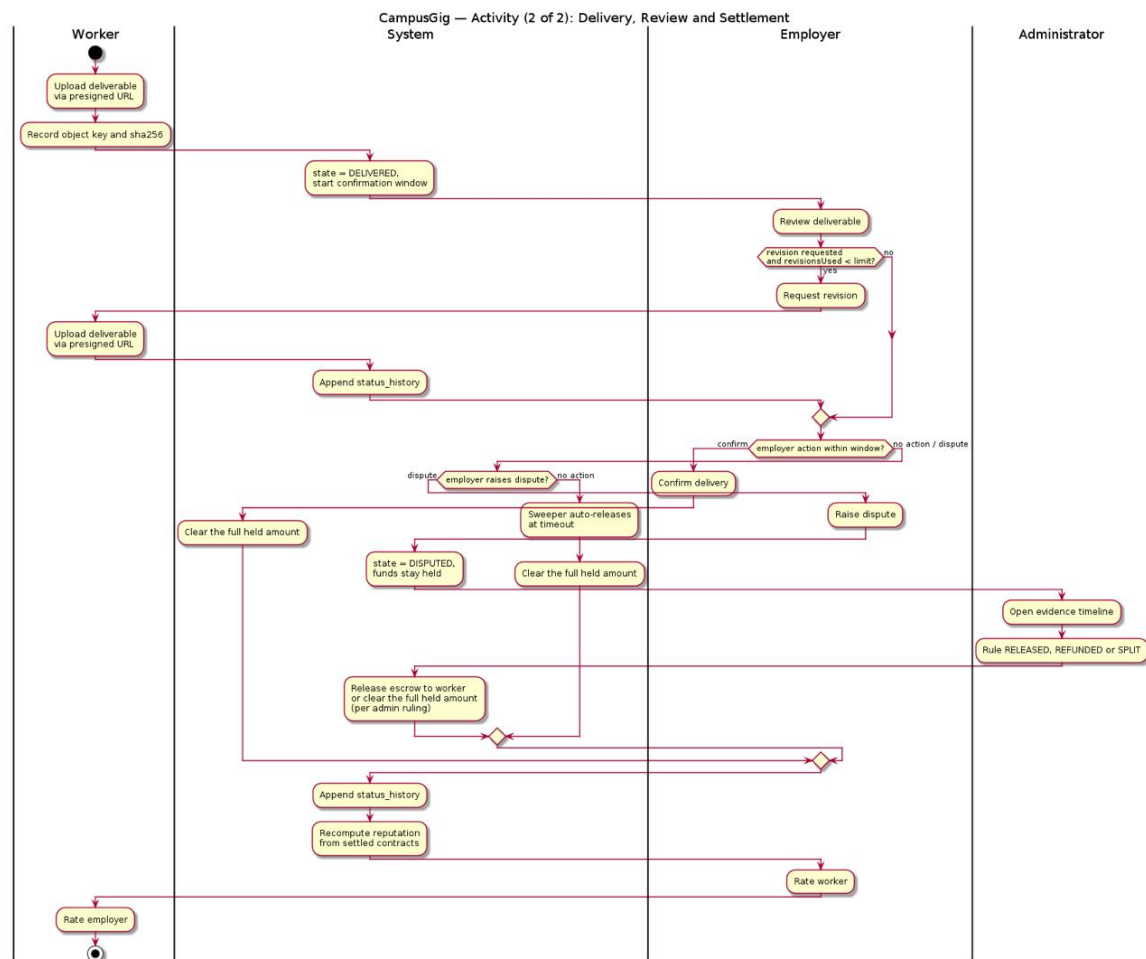


Figure 2.6. Activity diagram, part 2 of 2: delivery, review and settlement, including the dispute path.

This half is entirely design work. The worker uploads a deliverable, the system records its object key and SHA-256 and starts a confirmation window, the employer reviews and may request a revision within a limit, and settlement happens by one of three routes: the employer confirms, the sweeper auto-releases at timeout, or a dispute is raised and an administrator rules. Every route converges on clearing the full held amount, appending to the status history, and recomputing reputation. The only part of this that exists in code today is the single **COMPLETED** status a employer can set, which records that work was delivered but moves no money and stores no deliverable.

2.2.1 Module 1: Core Functionality

The first module covers identity, sessions and the gig board, which together are what makes the application usable at all.

Registration and the institutional rule. The @thapar.edu check lives

in `auth_service.assert_institutional_email`. It normalises the address with `strip().lower()`, splits on the last `@` using `rpartition`, and compares the whole domain for equality against the configured value. Suffix matching was rejected deliberately: `endswith("thapar.edu")` accepts `a@fakethapar.edu`, and no suffix rule at all rejects `a@thapar.edu.fake.com`. Whole-domain equality rejects both by construction, and the test suite asserts exactly those strings.

An email OTP verification step was built and then removed on request, with migration 0002 dropping the `email_verifications` table and the `users.is_verified` column. The consequence is stated plainly in the repository's architecture document and is worth repeating here: with verification gone, the domain check is the entire access control, and nothing proves that the person signing up controls the address they typed. This is an accepted risk rather than an oversight, and re-adding verification is a new migration plus a service rather than a redesign.

Registration deliberately does not create a session. It returns 201 and the client sends the student to the login page, so signing in is always an explicit act.

Sessions. The project uses an opaque server-side session token rather than a JWT, and the reason is a requirement rather than a preference. Logout has to invalidate the session, and a stateless JWT cannot be revoked without adding a denylist table, which is more machinery than simply storing the session in the first place. On login the service generates a 256-bit token with `secrets.token_urlsafe(32)`, stores only its SHA-256 in the `sessions` table, and returns the raw value in a cookie marked `HttpOnly`, `SameSite=Lax`, and `Secure` when the environment is production. Logout deletes the row. Expiry, revocation and logging out of every device all follow from that one design choice, and the raw token exists nowhere on the server. This follows the session management guidance published by OWASP [23b].

Authentication is enforced by one dependency, `get_current_user`, which reads the cookie, hashes it, loads a live session and returns the user or raises `NotAuthenticated`. A route opts in by declaring `CurrentUser` in its signature. No route body contains an authentication check, which is what makes it impossible to forget one.

Login hardening. Three measures sit around the password comparison. Passwords are hashed with bcrypt at cost factor 12. When the email does not match any account, the service still runs a bcrypt comparison against a dummy hash, so an unknown account and a wrong password take the same time and return an identical 401 body, which closes the account enumeration channel. Ten consecutive failures set `locked_until` fifteen minutes ahead and the endpoint answers 429 until it passes; a

successful login resets the counter.

The gig board. GET `/api/v1/gigs` returns a page of OPEN gigs. Filtering is assembled in `gig_repo`: a case-insensitive ILIKE search across title and description, a category equality filter, minimum and maximum budget bounds, a deadline upper bound, and one of four sorts. Pydantic validates every query parameter at the boundary, the service refuses an inverted budget range, and `page_size` is capped at 50 so no client can ask for the whole table. The response is a generic `Page[T]` envelope carrying `items`, `page`, `page_size`, `total` and `total_pages`, with the count computed by a separate query over the same filter conditions.

Posting a gig accepts only the fields a student may set. `GigCreate` contains no `employer_id` and no `state` field at all, so the employer is always the session user and a new gig is always OPEN. This is the same reasoning that keeps `role` out of the registration schema: a field that cannot be sent cannot be forged. Deadlines are validated to be in the future, with a naive timestamp treated as UTC rather than rejected.

One routing detail is worth recording because it is a real bug that ordering prevents. `/gigs/mine` is declared before `/gigs/{gig_id}` in the router, otherwise FastAPI would try to parse the literal string `mine` as a UUID and return a validation error.

The sequence diagram in Figure 2.7 shows how the acceptance step is designed to work once contracts and escrow exist.

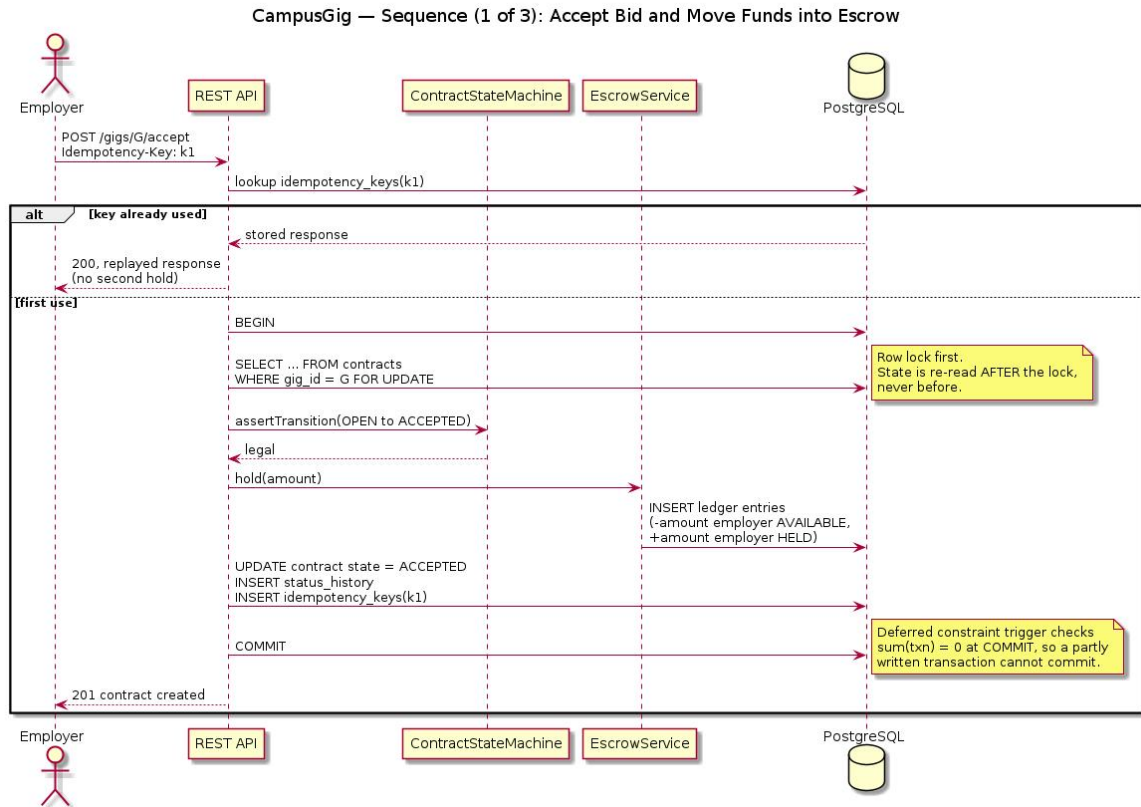


Figure 2.7. Sequence diagram, part 1 of 3: accepting a bid and moving funds into escrow, including the idempotency and row-locking steps.

Four things happen in a specific order and the order is the point. The API first looks up the idempotency key, and if it has been seen the stored response is replayed with no second hold, which is what makes a retried request safe. Otherwise a transaction begins, the contract row is locked with `SELECT ... FOR UPDATE`, and only after the lock is held is the state re-read and checked against the state machine, so two simultaneous acceptances cannot both pass the check. The escrow service then writes the balanced pair of ledger entries, the contract state and status history are updated, and at `COMMIT` the deferred constraint trigger verifies that the transaction sums to zero.

The implemented `set_status` function performs the equivalent domain checks: it confirms that the caller is the gig's employer, that the requested transition is legal, and it closes the gig and rejects the other pending applications in the same database transaction. What it does not do is take a row lock or write a ledger entry, which is the honest gap between Figure 2.7 and the current code.

2.2.2 Module 2: Secondary Features

The second module covers the applications workflow, the student profile, and file storage. These build on Module 1 and are what turn a gig board into something a student can actually complete a piece of work through.

Applications. An **Application** joins a gig to an applicant with an optional message and a status. Applying is refused in three cases, each with its own error code and status: the gig does not exist (404), the applicant is the employer (400), or the gig is not **OPEN** (409). Duplicate applications are prevented by a **UNIQUE** (**gig_id**, **applicant_id**) constraint in the database rather than by checking first and then inserting, because the check-then-insert version has a race window between the two statements. The service catches the resulting **IntegrityError** and converts it to an **AlreadyApplied** error, so the database is the authority and the API still returns a clean 409.

The employer-side actions run through **set_status**, which is described in Section 2.2.1. Accepting one applicant has two side effects in the same transaction: the gig moves to **CLOSED** so it stops appearing in the listing, and every other application still in **APPLIED** is set to **REJECTED**. This is what prevents a single gig from ending up with several accepted workers.

The Applied page is served by a single endpoint, **GET /api/v1/applications/me**, which returns both the summary counts and the list they describe. Combining them into one response is deliberate: two separate calls could return figures computed at different moments and disagree on screen. The summary includes an **earned** figure, which is the sum of gig budgets over the student's **COMPLETED** applications. This is the value of work completed and not money that has moved. No payment system exists in this release, and the page states that under the report cards rather than leaving the impression that the platform handled a payment.

Profile and file storage. A profile holds the details captured at signup, which are read-only, plus a short description, a profile picture and a resume. Uploads are handled by **storage_service**, which makes three decisions worth explaining [23a]. File type is determined by reading the first bytes of the file and matching them against known signatures for JPEG, PNG, GIF, WEBP and PDF, never from the **Content-Type** header the browser sends, which is entirely under the client's control. Size is enforced while the file streams in, aborting and deleting the partial file the moment the cap is exceeded, rather than accepting the whole upload and checking afterwards. And the stored filename is a fresh UUID with an extension derived from the detected type, so a client-supplied filename can never carry a path separator or a

second extension.

Replacing a file writes the new path to the database, commits, and only then deletes the old file. The order matters: if the delete fails, the result is a stray file on disk, which is harmless, whereas deleting first would risk a profile row pointing at a file that no longer exists.

Frontend structure. The browser application holds authentication state in an `AuthProvider` that calls `GET /auth/me` once on mount, which is what keeps a student signed in across a page refresh. It exposes a three-valued status of `loading`, `authenticated` or `anonymous`, and `ProtectedRoute` renders a spinner while the status is `loading`; without that, a refresh would briefly flash the login page before the session check answered. When any request returns 401, the API client dispatches a browser event that the provider listens for and drops the local user, so the interface cannot keep showing a signed-in state for a session the server has already discarded. The route guards are a convenience for the person using the application and are not a security control, because the backend rejects unauthenticated calls regardless of what the router decides.

The remaining two sequence diagrams document behaviour that has been designed but not built. Figure 2.8 covers delivery and the race between an employer confirming and the timeout sweeper firing.

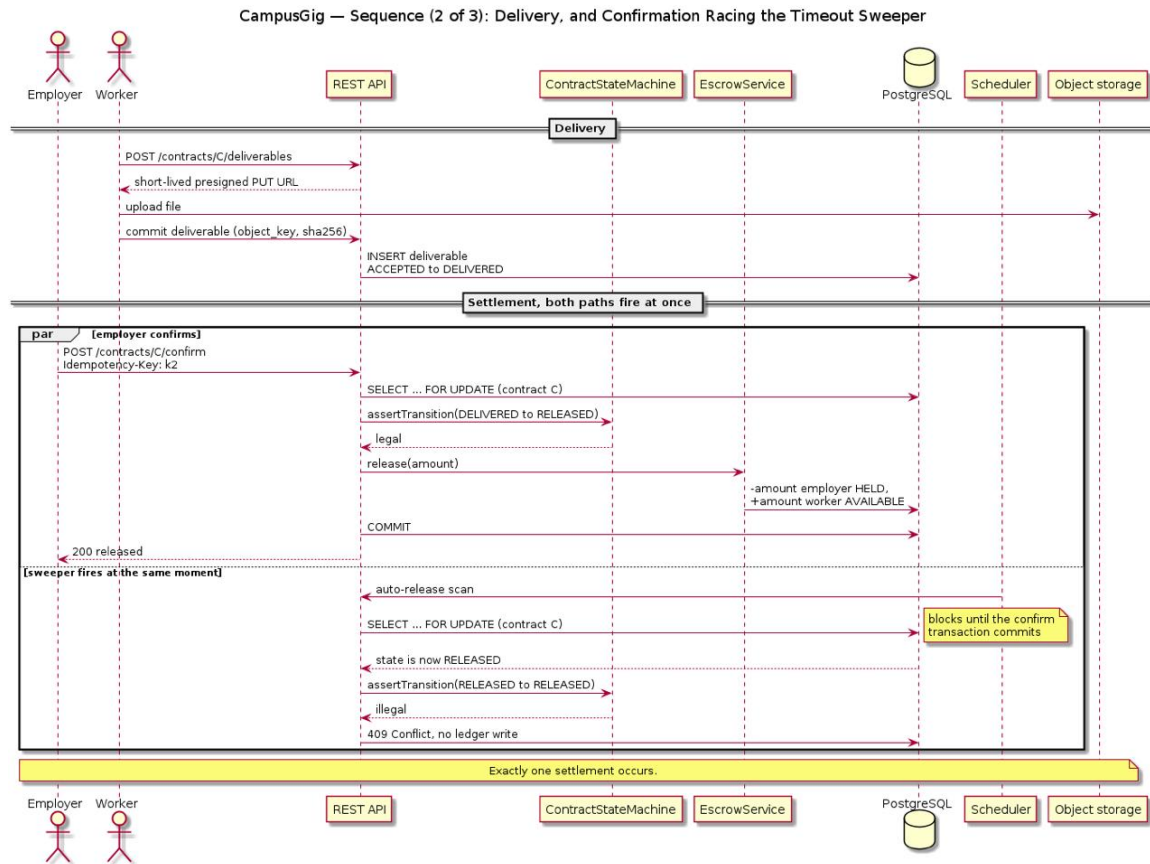


Figure 2.8. Sequence diagram, part 2 of 3: deliverable upload, and confirmation racing the auto-release sweeper.

The upload half uses a short-lived presigned URL so the file goes directly to object storage and the API only records the object key and hash. The settlement half is the interesting part. Both the employer’s confirmation and the scheduler’s auto-release scan are shown firing at the same instant. The confirming transaction takes the row lock first, so the sweeper’s `SELECT ...FOR UPDATE` blocks until that transaction commits; when it finally proceeds, the state it reads is already `RELEASED`, the state machine rejects a `RELEASED` to `RELEASED` transition, and the sweeper answers 409 without writing to the ledger. Exactly one settlement occurs, which is the property the design has to guarantee. The current implementation has neither a scheduler nor object storage, so this diagram is a specification rather than a description.

Figure 2.9 covers the dispute path.

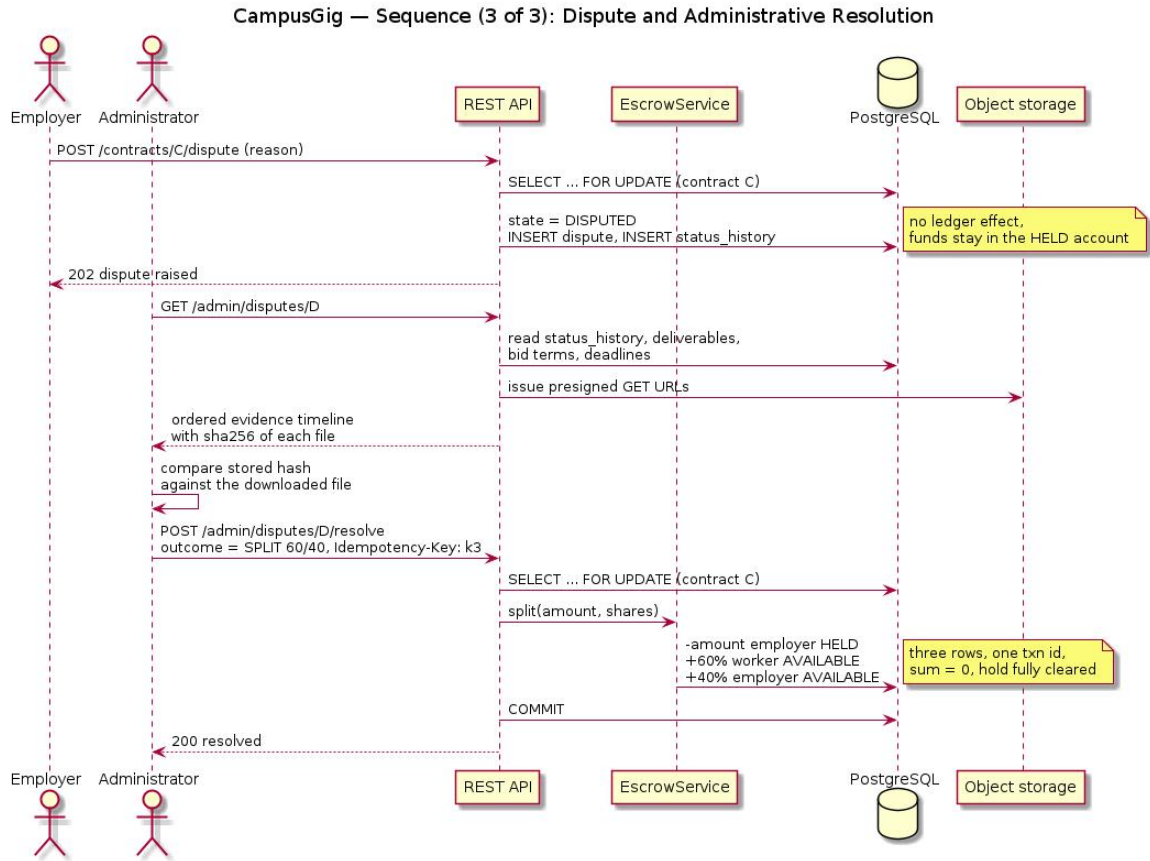


Figure 2.9. Sequence diagram, part 3 of 3: raising a dispute and resolving it from the evidence trail.

Raising a dispute has no ledger effect at all: the contract moves to **DISPUTED** and the funds simply stay in the held account. That is what makes the dispute state safe to enter, because nothing is paid out while it is being examined. The administrator's view is assembled from the status history, the deliverable records, the bid terms and the deadlines, and it includes the stored SHA-256 of each file so the administrator can confirm that the file they downloaded is the file that was uploaded. Resolution takes the same row lock and writes a single balanced transaction, which for a 60/40 split is three entries: one debit against the held account and two credits, summing to zero and clearing the hold completely.

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** Individual pure functions are tested directly. On the frontend, `validation.test.ts` exercises the institutional email rule and the password rule against a table of accepted and rejected inputs. On the backend, the model-level tests in `test_health_and_model.py` check column defaults, the database-generated `email_hash`, the case-insensitive uniqueness of `email`, the batch range check constraint, and the guard that refuses to start in production with the default `AUTH_SECRET`.
- **Integration Testing:** This is where most of the testing effort sits. The backend suite runs against a real PostgreSQL database, not an in-memory substitute, and `confest.py` creates a separate `<db>_test` database and applies the actual Alembic migrations to it before the suite starts. Testing against the migrated schema rather than against `metadata.create_all()` means that a migration which does not match the models is caught by the tests instead of at deployment. Tables are truncated between tests so cases do not leak state into one another. Requests go through FastAPI's `TestClient`, so a test exercises the full path from route through service and repository to the database and back.
- **Performance Testing:** None was carried out. No load generator was run, no response times were recorded, and no throughput figure was measured, so this report contains no performance numbers. Section 3.2 reports what was actually measured instead.
- **User Testing:** No formal user study was conducted. Validation was done by the team running the full flow manually against a live backend and a live PostgreSQL instance: registering, logging in, posting a gig, browsing and filtering, applying from a second account, accepting from the first, and logging out. The development seed script, `app.seeds.seed_gigs`, populates gig data for this and refuses to run when the environment is production.

The suite is deliberately weighted towards rejection paths. Roughly two thirds

of the backend cases assert that something is refused, with the correct status code and the correct error code, rather than asserting that a happy path works. Accepting a valid input is the easy half of an API; refusing an invalid one correctly is where the defects are.

3.2 Performance Metrics

No performance benchmarking was performed, so there are no response time, accuracy or throughput results to report. What can be reported honestly is the size and composition of the automated test suite, and the status of each stated objective. Table 3.1 gives the test counts by area, taken from the test files in the repository.

Area	Test File	Cases
Registration	<code>test_registration.py</code>	29
Login and sessions	<code>test_auth_session.py</code>	15
Gig listing and detail	<code>test_gigs.py</code>	16
Posting and applications	<code>test_applications.py</code>	22
Profile and uploads	<code>test_profile.py</code>	12
Health and model	<code>test_health_and_model.py</code>	9
Backend total	<code>pytest</code>	103
Validation rules	<code>validation.test.ts</code>	16
Auth forms and guards	<code>auth.test.tsx</code>	4
Frontend total	<code>vitest</code>	20

Table 3.1. Automated test cases by area, counted from the repository. Parameterised cases are counted individually.

Table 3.2 states the status of each objective from Chapter 1 against the code that exists in the repository.

Objective	Status	Evidence
1. Institutional access	Met	<code>auth_service</code> , 9 domain tests
2. Revocable sessions	Met	<code>sessions</code> table, 15 tests
3. Gig board	Met	<code>gigs.py</code> , 16 tests
4. Application workflow	Met	<code>application_service</code> , 22 tests
5. Profile and uploads	Met	<code>storage_service</code> , 12 tests
6. UML design	Met	9 diagrams, Chapter 2
7. Rejection paths tested	Met	123 cases, majority negative

Table 3.2. Objective status against the current implementation

For scale, the backend application code is approximately 2,100 lines of Python across 18 route handlers, with a further 1,167 lines of tests and 345 lines of Alembic migrations. The frontend is approximately 3,250 lines of TypeScript and TSX with 199 lines of tests. The schema is defined by three sequential migrations and four tables: `users`, `sessions`, `gigs` and `applications`.

3.3 Analysis

- **Did you meet your objectives?** All seven objectives stated in Chapter 1 were met, but those objectives were scoped to the first three milestones of the eight in the project plan. Against the full platform design shown in Chapter 2, the implementation covers identity, gigs, applications and profiles, and does not cover contracts, escrow, the ledger, deliverable upload, disputes, ratings or admin tooling. The project is a working system, not a finished one.
- **Where did you exceed expectations?** Three areas ended up stronger than planned. The test suite grew well past the original target, and running it against real migrations rather than a synthesised schema turned out to be worth the extra setup. The error handling ended up fully unified, so every failure in the system reaches the browser in one shape with a stable machine-readable code. And several checks that could have lived in application code were pushed into the database instead: the unique constraint on (`gig_id`, `applicant_id`), the case-insensitive `citext` email column, the generated `email_hash` column, and the check constraints on title length, budget and batch year. Those cannot be bypassed by a code path that forgets to check.
- **What challenges did you encounter?** The first was choosing a session mechanism. A JWT was the obvious default but could not satisfy the requirement that logout actually invalidate a session without adding a denylist table, at which point the stateless argument for a JWT disappears. The second was the domain check, where the naive `endswith` implementation passes a casual review and still accepts `a@fakethapar.edu`. The third was preventing duplicate applications correctly, where the obvious check-then-insert has a race window. The fourth was file upload validation, where trusting the `Content-Type` header is both easy and wrong. The fifth was scope control: the UML design describes a full escrow platform, and the temptation to put a payment button in the interface before there was a ledger behind it was real.
- **How did you resolve them?** The session problem was resolved by storing a hashed opaque token server-side, which makes logout a row deletion. The domain

check became a whole-domain equality comparison after splitting on the last `@`, with the specific attack strings written into the test suite so a future change cannot quietly reintroduce the bug. Duplicate applications were pushed down to a database unique constraint with the resulting integrity error translated into a 409. File type detection was moved to magic byte inspection with size enforced during streaming. Scope was controlled by the rule recorded in the project plan: nothing from an unimplemented milestone appears in the interface, and the one place where a money-shaped figure is shown, the earned total on the Applied page, carries a statement on the page that no payment has taken place.

The most significant limitation is that the platform solves the coordination problem but not the payment problem, which was the fourth problem identified in Section 1.1.1. Two other limitations are worth recording. Uploaded files live on local disk, which is correct for one process and wrong the moment a second instance exists, because the two would not share a filesystem. And uploaded files are served from unguessable UUID paths with no access check, so the URL itself is the capability; that is acceptable for a profile picture and would not be acceptable for a private deliverable.

CHAPTER 4

Conclusion

4.1 Summary of Work

The objective set out in Chapter 1 was to build a gig marketplace restricted to a single institution, to design the complete platform in UML, and to back the implemented parts with automated tests. All three were achieved for the scope defined by the first three milestones.

What runs today is a complete application. A student with an `@thapar.edu` address can register, log in to a server-side session held in an `HttpOnly` cookie, and log out in a way that invalidates that session on the server. Behind the login there are three pages. The gigs board lists open postings with search, category, budget and deadline filters, four sort orders and pagination, and a floating button that posts a new gig. The applied page lists every gig the student applied to along with summary counts and the value of completed and in-progress work. The profile page holds a picture, a resume in PDF form, a short description and the read-only signup details. A gig's detail page lets a student apply with a message, or, if the gig is theirs, shows the applicants and lets them accept, decline or mark the work complete.

The design work went considerably further than the implementation, and this was intentional. Nine UML diagrams model the full platform including contracts, double-entry escrow, deliverable evidence and dispute resolution. Objectives 1 through 5 and 7 describe the implemented system, and Objective 6 describes the design that the remaining five milestones will be built against.

4.2 Key Findings

- **Finding 1:** Constraints are more reliable in the database than in application code. Every rule that could reasonably be pushed into PostgreSQL was, including the unique constraint that prevents duplicate applications, the `citext` column that makes email uniqueness case-insensitive, and the generated column that derives `email_hash` from `email`. Each of these removes a class of bug rather than a single instance of it, because no code path can forget to apply them.

- **Finding 2:** A security check that reads correctly can still be wrong. The email domain rule is the clearest example in the project: `endswith("thapar.edu")` looks like a correct implementation of the requirement and accepts `a@fakethapar.edu`. Writing the specific attack strings into the test suite was worth more than reviewing the line of code, because the test states the requirement in a form that cannot be misread later.
- **Finding 3:** Requirements decide architecture more often than preferences do. The session design is a case in point. A stateless JWT was the default expectation, but the requirement that logout invalidate the session made it strictly more complicated than storing the session, because a denylist table would have been needed anyway.
- **Finding 4:** Designing further ahead than the implementation was productive rather than wasted. Modelling the contract state machine and the ledger before writing the application status code is why `ALLOWED_TRANSITIONS` has the same shape as the state diagram, and why the escrow milestone can extend the existing code instead of replacing it.

4.3 Future Work and Recommendations

1. Implement Milestone 4, contracts. This means turning an accepted application into a contract row with the state machine of Figure 2.4, a status history table, and guards that reject an illegal transition with a 409 [Fow03].
2. Implement Milestone 5, escrow and the ledger. This is the highest-value remaining work because it is what turns the platform from a coordination tool into one that protects both sides of a transaction. It requires the account and ledger tables from Figure 2.3, the deferrable sum-to-zero constraint trigger, idempotency keys on every money-moving endpoint, and row locking on the acceptance path as shown in Figure 2.7.
3. Replace local disk storage with an object store such as S3 or MinIO. This is a prerequisite for running more than one backend instance and for the presigned upload flow in Figure 2.8. Because `storage_service` is the only module that touches the filesystem, the change is contained to one file.
4. Add the operational controls that were deliberately deferred: per-IP rate limiting, which needs a shared store such as Redis, an audit log, and virus scanning of uploads.

5. Reconsider email verification. Removing it was a scope decision, and the domain check is currently doing all of the access control work alone. Reinstating an OTP step would restore proof that the person controls the address they registered with.
6. Move gig search from an `ILIKE` scan to a PostgreSQL full-text index. The current approach is appropriate at the present table size and should be revisited once the gig table grows into the thousands of rows.
7. Add ratings and reputation, Milestone 7, which is what would let a employer judge an applicant they do not personally know.

4.4 Final Remarks

The clearest lesson from this project was how much of software engineering is deciding what not to build. The UML design describes a platform with escrow, a double-entry ledger, dispute resolution and reputation. It would have been straightforward to put a payment button in the interface and a placeholder behind it, and the application would have looked more complete during a demonstration. The project plan in the repository rules that out explicitly, and the one place where a money-shaped number appears on screen carries a note saying that no payment has occurred. Keeping that line held is the decision the team is most confident was correct.

The second lesson concerns where correctness belongs. Repeatedly, the right answer was to move a rule somewhere it could not be bypassed: into a database constraint rather than a service check, into a single authentication dependency rather than a check in each route, into a schema that omits a field rather than a validator that rejects it. A field that cannot be sent cannot be forged, and that turned out to be a more useful principle than any amount of defensive validation.

Finally, writing the tests against real PostgreSQL with the real migrations applied cost noticeably more setup effort than an in-memory database would have, and it repaid that cost. Several of the guarantees this report describes, the generated email hash, the case-insensitive uniqueness, the unique constraint on applications, exist only in the migrations. A test suite that never ran those migrations would have reported all of them as working while proving nothing.

Bibliography

- [Bay24] Michael Bayer. *SQLAlchemy 2.0 Documentation*. SQLAlchemy Project. 2024 (cit. on p. 10).
- [23a] *File Upload Cheat Sheet*. Open Worldwide Application Security Project. 2023 (cit. on p. 16).
- [Fow02] Martin Fowler. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002 (cit. on p. 5).
- [Fow03] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd ed. Boston: Addison-Wesley, 2003 (cit. on p. 25).
- [24a] *PostgreSQL 17 Documentation*. The PostgreSQL Global Development Group. 2024 (cit. on p. 10).
- [PM99] Niels Provos and David Mazières. “A Future-Adaptable Password Scheme”. In: *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*. USENIX Association, 1999, pp. 81–91 (cit. on p. 10).
- [Ram24] Sebastián Ramírez. *FastAPI Documentation*. FastAPI. 2024 (cit. on p. 10).
- [24b] *React Documentation*. Meta Open Source. 2024 (cit. on p. 10).
- [23b] *Session Management Cheat Sheet*. Open Worldwide Application Security Project. 2023 (cit. on p. 13).